



Encontro 4

24/09/2026 | Containers, integração e troubleshooting

Do programa ao ambiente integrado

Encontro 4 • entender os componentes antes de executar comandos

Entender

Aplicação, processo e runtime.

Container, imagem e Docker.
Compose e comparação com VMs.

Observar

Executar um container simples.

Construir a imagem da API.
Integrar API e banco.

Aplicar

Diagnosticar uma falha.
Containerizar o projeto da equipe.
Guardar evidências para a entrega final.

Resultado: outra pessoa consegue subir o projeto, testar seu funcionamento e entender sua configuração.

Roteiro das quatro horas

Exposição progressiva, demonstração e prática no projeto

0–65 min | Base

0–10: objetivo e contexto.

10–50: conceitos e comparação.

50–65: primeira demonstração e verificação.

65–165 min | Integração

65–95: imagem da API.

95–110: intervalo.

110–145: Compose e integração.

145–165: diagnóstico guiado.

165–240 min | Projeto

165–235: laboratório das equipes.

235–240: fechamento e evidências.

Consultas ficam disponíveis após a trilha.

As verificações vêm depois da explicação; as equipes aplicam os conceitos ao próprio projeto.

Uma aplicação precisa de mais que o código

Exemplo: uma API de visitas escrita em JavaScript

Código e bibliotecas

server.js define as rotas.

O pacote pg permite consultar PostgreSQL.

Os arquivos precisam estar disponíveis.

Ambiente de execução

Node.js executa o JavaScript.

Bibliotecas do sistema dão suporte.

O sistema operacional oferece recursos.

Configuração e serviços

HOST e DATABASE_URL orientam a execução.

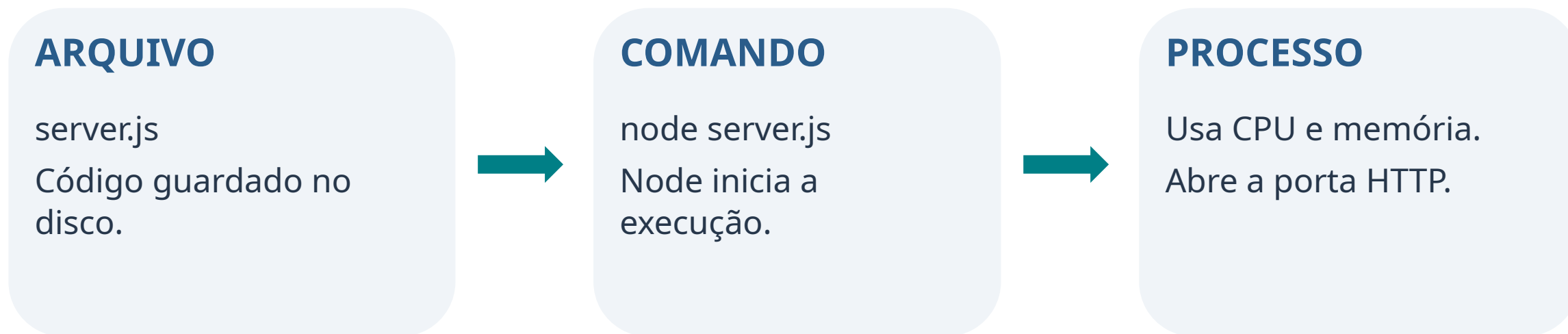
A API precisa alcançar o banco.

Os dados precisam de armazenamento.

A execução depende da combinação código + runtime + dependências + configuração + serviços.

Programa é arquivo; processo é execução

Ao iniciar o programa, o sistema operacional cria uma atividade em execução



O arquivo permanece; o processo pode iniciar e terminar.

Um container em execução terá um processo principal; quando esse processo termina, o container para.

Sistema operacional: recursos para os processos

Vocabulário mínimo para entender o isolamento

Kernel

Núcleo do sistema operacional.

Gerencia CPU, memória, arquivos e rede.

Atende solicitações dos processos.

Espaço de usuário

Aplicações, comandos e bibliotecas.

Node.js e a API executam aqui.

Usam serviços oferecidos pelo kernel.

Exemplo concreto

A API lê um arquivo e abre uma conexão.

O processo solicita essas operações.

O kernel controla os recursos envolvidos.

Compartilhar o kernel não significa compartilhar todos os arquivos, processos visíveis ou endereços de rede.

Runtime da aplicação: quem executa o programa

Runtime é o ambiente necessário para executar o código da aplicação

Aplicação	Runtime / execução	Exemplo
JavaScript no servidor	Node.js	node server.js
Python	Interpretador Python	python app.py
Java	JVM	java -jar app.jar

No laboratório, Node.js é o runtime da API; pg é uma biblioteca; PostgreSQL é outro serviço.

Container: processo em um ambiente isolado

Definição operacional para os exemplos desta aula

O que é

Uma instância com processos, arquivos e configuração próprios, criada a partir de uma imagem.

O que faz

Permite executar aplicações com dependências separadas e uma configuração de execução definida.

Como funciona

Usa recursos do kernel para separar visibilidade e controlar acesso a recursos. O processo continua executando no sistema.

Empacotar a aplicação e executá-la em um container reduz diferenças de ambiente; não elimina todas elas.

Isolamento: o que fica separado

Mecanismos do Linux usados na execução de containers

Visibilidade

Namespaces podem separar a visão de processos, rede e pontos de montagem.

A API tem sua própria perspectiva de localhost.

Recursos

Cgroups permitem contabilizar e limitar CPU e memória.

Um limite precisa ser configurado; não presume um valor automático.

Arquivos e permissões

A imagem fornece arquivos-base.

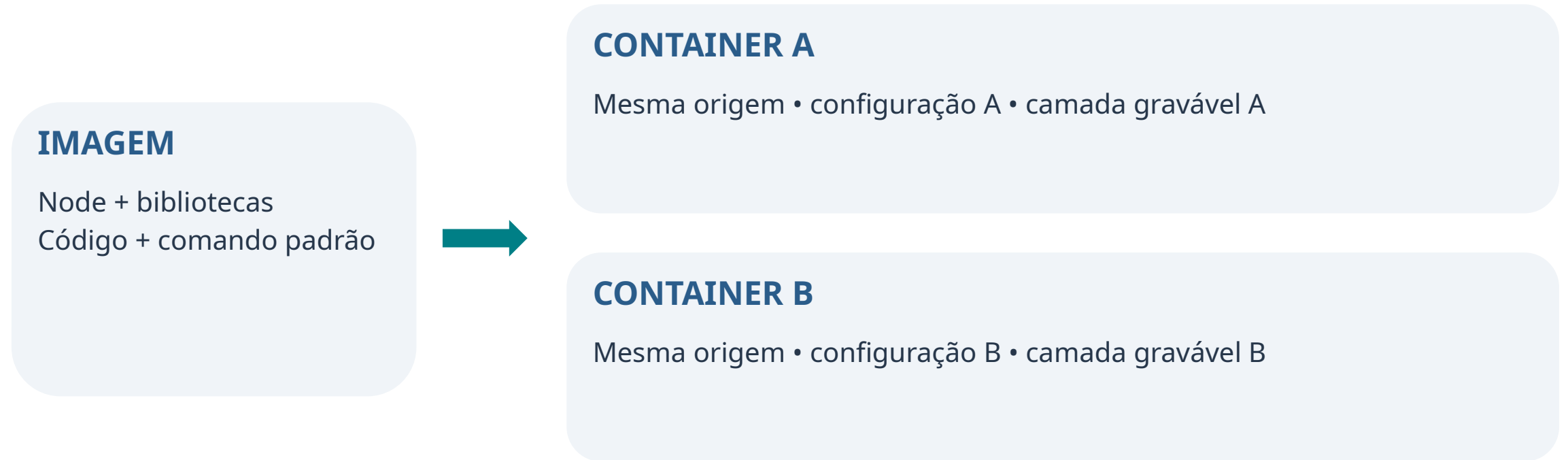
A instância recebe uma camada gravável.

Usuário e permissões também importam.

Isolamento, limites e persistência são escolhas configuradas; o nome “container” não define sozinho essas políticas.

Imagem e container são objetos diferentes

Artefato de origem versus instância com estado



Um novo container recebe a imagem e a configuração escolhidas; não herda automaticamente edições de outra instância.

Runtime de containers: quem prepara o isolamento

A palavra runtime também aparece em outra camada

RUNTIME DE CONTAINERS

containerd gerencia o ciclo de vida.
runc prepara e inicia o processo isolado.
Atuam na execução do container.

RUNTIME DA APLICAÇÃO

Node.js executa server.js.
A API usa Node dentro da instância.
Atua na execução do código.

Node.js e um runtime de containers não se substituem: atuam em camadas diferentes.

Docker: ferramentas para trabalhar com containers

Construir imagens, distribuir artefatos e gerenciar a execução

Docker CLI

O comando docker é o cliente.

Recebe sua solicitação no terminal.

Envia operações à API do Engine.

Docker Engine

Serviço que gerencia imagens, containers, redes e volumes.

No Linux, o daemon é o dockerd.

Ecossistema

BuildKit participa da construção.

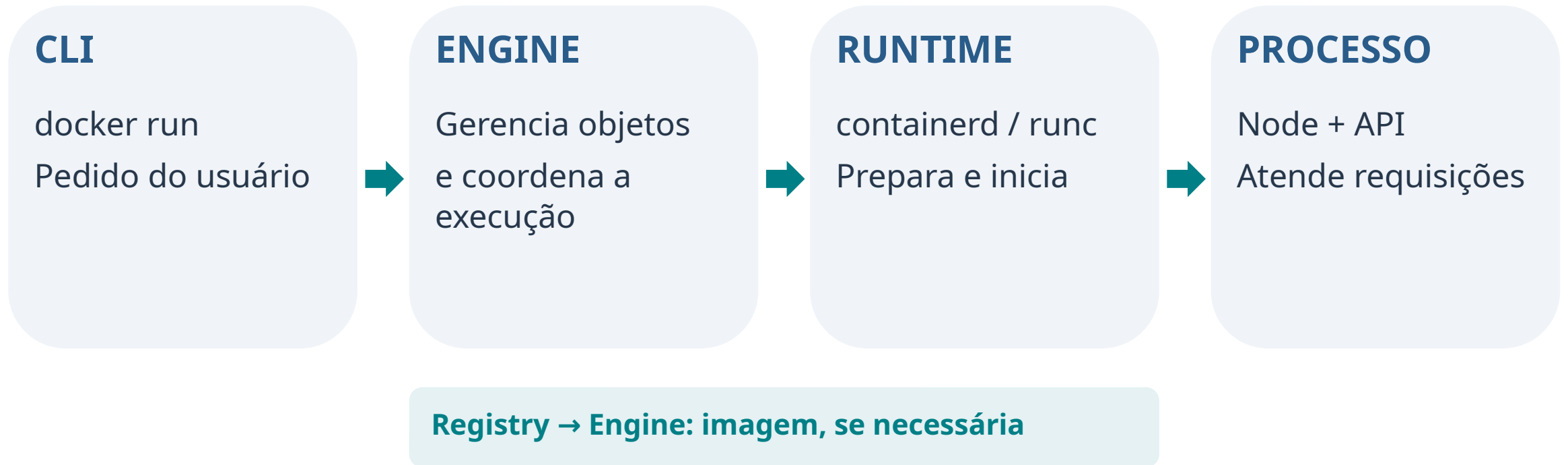
Registries armazenam imagens.

Docker Desktop facilita o ambiente local.

Docker é a ferramenta de trabalho; imagem é o artefato; container é a instância executável.

O caminho de um docker run

Visão simplificada do pedido até a execução



O terminal controla a execução; o registry distribui a imagem; os processos executam no host do Engine.

Engine e Desktop: onde o container roda

Nesta aula trabalhamos com containers Linux

Linux com Engine

O Engine pode executar sobre o Linux do host.

Os containers compartilham esse kernel.

Docker Desktop

Oferece uma instalação integrada.

Containers Linux usam um ambiente Linux virtualizado. No Windows, pode usar WSL 2.

Consequência

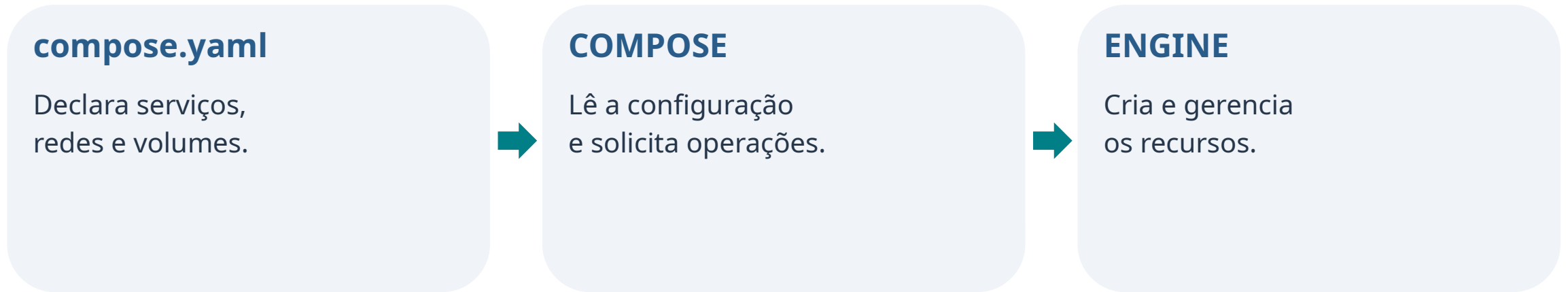
Não há um kernel exclusivo em cada imagem.

Arquitetura de CPU e compatibilidade da imagem ainda precisam ser consideradas.

VMs e containers podem coexistir: uma VM Linux pode hospedar vários containers.

Docker Compose: descrever a aplicação integrada

Um arquivo declara serviços e os recursos usados em conjunto



Resultado do exemplo: app + db + rede compartilhada + volume dados

Dockerfile descreve como construir uma imagem; compose.yaml descreve como executar e integrar os serviços.

VM e container: compare a estrutura

Agora que os componentes estão definidos, observe onde cada kernel fica

MÁQUINAS VIRTUAIS

Aplicação + libs
SO + kernel
convidado

Aplicação + libs
SO + kernel
convidado

Hypervisor

Host / hardware

CONTAINERS LINUX

Aplicação + libs
Processo isolado

Aplicação + libs
Processo isolado

Runtime de containers

Kernel do host Linux

VM virtualiza uma máquina; container organiza processos sob isolamento do sistema operacional.

VM e container: diferenças que orientam a escolha

Compare requisitos; evite a regra “um sempre substitui o outro”

Critério	Máquina virtual	Container Linux
Kernel	Cada VM possui seu kernel convidado.	Compartilha o kernel do host Linux.
Conteúdo	Inclui um SO convidado a administrar.	Imagem reúne os arquivos de usuário necessários.
Uso típico	Outro SO/kernel; separação no nível da VM.	Empacotar e executar componentes de aplicação.
Combinação	Pode hospedar vários containers.	Pode rodar em um host físico ou virtual.

Containerizar um monólito não o transforma em microsserviços: arquitetura e empacotamento são decisões diferentes.

O que a containerização resolve — e seus limites

Redução de diferenças de ambiente com responsabilidades explícitas

Padroniza

Runtime e bibliotecas escolhidos.
Arquivos presentes na imagem.
Comando e configuração declarados.

Ainda depende

Kernel e plataforma compatíveis.
Recursos disponíveis e rede.
Credenciais, serviços externos e dados.

Não fornece sozinha

Correção de bugs.
Backup e alta disponibilidade.
Segurança absoluta ou arquitetura de microsserviços.

A execução confiável combina artefato conhecido, configuração correta, recursos adequados e verificação.

Mapa dos recursos do Docker

Cada recurso resolve uma necessidade diferente da aplicação

Necessidade	Recurso	Exemplo na aula
Empacotar e distribuir	Imagens, build e registry	API com Node e dependências.
Executar e controlar	Containers e ciclo de vida	Criar, iniciar, parar e remover.
Conectar e guardar	Redes, portas e volumes	API acessa db; contador persiste.
Observar e restringir	Logs, inspect, stats e limites	Investigar falhas e consumo.
Integrar componentes	Compose e healthchecks	Declarar e verificar a aplicação.

Escolha o recurso pelo problema: imagem não guarda o banco; volume não publica uma porta.

Primeira demonstração: um comando que termina

Observar o runtime da linguagem dentro de uma imagem pronta

```
docker version  
docker run --rm node:22-bookworm-slim node --version
```

Antes de executar

Engine/Desktop ativo.

Acesso ao registry se a imagem não estiver local.

Node no host não é obrigatório.

O que acontece

Docker cria a instância.

Node imprime uma versão 22.x.

O processo termina; --rm remove a instância.

O container não precisa permanecer ativo: sua duração acompanha o processo principal.

Segunda demonstração: um servidor que permanece ativo

Exemplo isolado • terminal Bash/zsh • porta local 18080 livre

```
docker run -d --name e4-primeiro \
  -p 127.0.0.1:18080:3000 \
  node:22-bookworm-slim node -e '
    require("node:http").createServer((req, res) => {
      res.end("Olá, turma!\n");
    }).listen(3000, "0.0.0.0");
  '
curl --retry 5 --retry-connrefused --retry-delay 1 \
  -fsS http://localhost:18080
```

Entenda o comando

-d: executa em segundo plano.
--name: nome da instância.
-p: host 18080 → container 3000.

Resultado esperado

A chamada HTTP retorna “Olá, turma!”.
O processo Node continua esperando novas requisições.

A porta publicada é uma entrada para chegar ao processo; ela não cria um servidor por si só.

Observar e executar comandos em uma instância

Aplicar enquanto e4-primeiro está em execução, antes de removê-lo

Comando	O que faz / resultado esperado
<code>docker logs --tail=20 e4-primeiro</code>	Mostra stdout/stderr; sem log no código, pode ficar vazio.
<code>docker exec e4-primeiro node --version</code>	Inicia outro processo na mesma instância; imprime 22.x.
<code>docker inspect --format '{{.State.Status}}' e4-primeiro</code>	Consulta metadados; neste momento: running.
<code>docker stats --no-stream e4-primeiro</code>	Mostra uma amostra de CPU, memória e I/O.

logs mostra o que foi emitido; inspect mostra configuração/estado; exec executa; stats mede consumo.

Ciclo de vida: criar, observar, parar e remover

Comandos aplicados somente ao container e4-primeiro da demonstração

Comando	O que observar
<code>docker ps / docker ps -a</code>	Instâncias ativas / todas as instâncias.
<code>docker top e4-primeiro</code>	O processo Node executando na instância.
<code>docker stop e4-primeiro</code>	Servidor para; a instância continua existindo.
<code>docker start e4-primeiro</code>	A mesma instância volta a executar.
<code>docker stop e4-primeiro</code> <code>docker rm e4-primeiro</code>	Encerra e remove somente a instância de demonstração.

Imagem, instância e processo têm ciclos de vida relacionados, mas não são o mesmo objeto.

Verificação | Componentes e responsabilidades

Verificação do bloco • responda com os conceitos e exemplos que acabamos de estudar

1. Situação

Em node server.js dentro de um container, qual é o papel de Node e qual é o papel do runtime de containers?

2. Situação

Duas instâncias criadas da mesma imagem são o mesmo container?

3. Situação

Compose instala um kernel e executa o JavaScript da aplicação?

Discuta primeiro. O próximo slide traz as respostas comentadas.

Respostas comentadas | Componentes e responsabilidades

Gabarito explicado • relacione cada resposta ao exemplo observado

1. Resposta

Node executa o JavaScript. O runtime de containers prepara o isolamento e inicia o processo nesse ambiente. São camadas diferentes.

2. Resposta

Não. Compartilham a origem, mas possuem identidade, configuração e estado de instância próprios.

3. Resposta

Não. Compose declara e coordena serviços usando o Engine. O processo usa seu runtime de linguagem e o kernel do host Linux.

Uma resposta completa identifica o componente envolvido e explica a evidência.

Agora vamos empacotar uma aplicação real

A mesma API de visitas vai ligar os conceitos de imagem, rede e dados

Aplicação

server.js em Node.js.
GET /visitas lê o total.
POST /visitas incrementa o total.

Dependência

PostgreSQL guarda o contador.
A biblioteca pg envia consultas.
DATABASE_URL informa o destino.

Contrato de saúde

/health/live: processo responde.
/health/ready: SELECT 1 no banco.
A operação de visitas será testada à parte.

A aplicação de exemplo ensina o padrão; a equipe deve adaptar esse padrão à aplicação que já escolheu.

Arquivos do laboratório: uma responsabilidade por arquivo

Local: Práticas/encontro-4-containers

Arquivo	Responsabilidade
server.js	Comportamento HTTP e acesso ao banco.
package.json + lockfile	Dependências e versões resolvidas.
Dockerfile + .dockerignore	Construção da imagem e controle do contexto.
compose.yaml	Execução dos serviços e recursos compartilhados.
init.sql + README.md	Inicialização didática do banco e instruções verificáveis.

Para reproduzir a execução, o repositório precisa trazer todos os arquivos necessários, além das instruções.

Dockerfile: a receita de construção da imagem

Arquivo completo da API • o código é JavaScript, sem etapa de compilação

```
FROM node:22-bookworm-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY --chown=node:node server.js ./
ENV NODE_ENV=production
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Durante o build

Selecionar a base.
Instalar pacotes pelo lockfile.
Copiar os arquivos da aplicação.

Na execução

Definir ambiente e usuário.
Documentar a porta.
Indicar o comando principal.

Build produz a imagem. A API começa a atender somente quando um container executa o comando definido.

Dockerfile: o que ocorre durante o build

A ordem cria os arquivos que farão parte da imagem

Instrução	Efeito no exemplo
FROM node:22-bookworm-slim	Escolhe o runtime e os arquivos-base.
WORKDIR /app	Define /app como diretório de trabalho.
COPY package.json package-lock.json ./	Coloca os manifests antes da instalação.
RUN npm ci --omit=dev	Instala as dependências de execução.
COPY --chown=node:node server.js ./	Inclui o código com propriedade definida.

O código e as dependências precisam entrar na imagem pelo build, não por edição manual de uma instância.

Dockerfile: como a instância vai iniciar

Metadados que orientam a execução do processo

Instrução	Efeito na execução
ENV NODE_ENV=production	Define um valor padrão de configuração.
USER node	Executa como o usuário node da imagem.
EXPOSE 3000	Documenta a porta; não publica no host.
CMD ["node", "server.js"]	Inicia o processo principal da API.
RUN executa uma etapa de construção; CMD define o comando padrão para a execução do container.	

Lockfile e cache: reproduzir sem reinstalar à toa

Instalar dependências antes de copiar o código favorece a reutilização

Ordem que desperdiça cache

```
COPY . .
```

```
RUN npm ci
```

Código mudou → reinstala

Ordem do laboratório

```
COPY package*.json ./
```

```
RUN npm ci
```

```
COPY server.js ./
```

Alteração de código não precisa reinstalar dependências quando suas entradas e o cache permanecem válidos.

Contexto de build e .dockerignore

O ponto em docker build indica o diretório usado como contexto

```
node_modules
.git
.env
.env.*
*.log
README.md
```

Duas fronteiras

Git filtra o versionamento.
Docker filtra o contexto.
.gitignore $\neq$.dockerignore.

No laboratório

node_modules local não entra.
Arquivos .env ficam fora.
O lockfile precisa continuar disponível.

Reprodução e segurança começam no conjunto de arquivos que você permite entrar na construção.

Construir a API e observar sua execução isolada

Executar dentro da pasta do laboratório • usar dois terminais

```
docker build -t e4-api:local .  
docker run --rm -p 127.0.0.1:18081:3000 e4-api:local  
  
# Em outro terminal:  
curl -i http://localhost:18081/health/live  
curl -i http://localhost:18081/health/ready
```

Terminal 1

Build cria e4-api:local.
run inicia a API em primeiro plano.
Ctrl+C encerra a demonstração.

Terminal 2

live responde 200.
ready responde 503 sem o banco.
Isso separa empacotamento de
integração.

Uma imagem construída com sucesso ainda precisa ser executada e testada no contexto de suas dependências.

Recursos de imagem: construir, obter e identificar

Comandos de consulta depois do build de e4-api:local

Operação	Comando / efeito
Construir	<code>docker build -t e4-api:local .</code> → produz a imagem do projeto.
Obter	<code>docker pull node:22-bookworm-slim</code> → obtém a imagem-base.
Listar	<code>docker image ls</code> → mostra referências locais.
Examinar	<code>docker image history e4-api:local</code> → histórico de construção.
Dar outra tag	<code>docker tag e4-api:local e4-api:revisao</code> → outra referência local.

build produz; pull obtém; tag referencia; push distribui; run cria uma instância.

Verificação | Imagem e execução

Verificação do bloco • responda com os conceitos e exemplos que acabamos de estudar

1. Situação

Trocar RUN npm ci por CMD npm ci mantém o mesmo comportamento?

2. Situação

Alterar server.js depois da cópia dos manifests precisa invalidar npm ci em todo build?

3. Situação

Se live retorna 200 e ready retorna 503 nesta execução isolada, o que já foi comprovado?

Discuta primeiro. O próximo slide traz as respostas comentadas.

Respostas comentadas | Imagem e execução

Gabarito explicado • relacione cada resposta ao exemplo observado

1. Resposta

Não. RUN instala durante o build; CMD executaria a instalação na partida, no lugar do comando principal esperado.

2. Resposta

Não necessariamente. A instalação pode usar cache se suas entradas e a base continuam válidas. O código foi copiado depois.

3. Resposta

O processo HTTP responde. A consulta de prontidão não passou; sem o banco integrado, ainda não há prova da funcionalidade de visitas.

Uma resposta completa identifica o componente envolvido e explica a evidência.

Compose: declarar o serviço da API

Serviço = definição • container = instância criada dessa definição

```
services:
  app:
    build: .
    ports:
      - "127.0.0.1:8080:3000"
    environment:
      DATABASE_URL: postgres://aula:aula@db:5432/aula
      HOST: 0.0.0.0
# Trecho: dependência e saúde aparecem adiante.
```

Construção e acesso

build: . aponta para o contexto.
ports publica 8080 → 3000.
O acesso do host fica local.

Configuração

DATABASE_URL aponta para db.
HOST orienta a interface de escuta.
Credenciais aqui são fictícias.

O arquivo permite revisar e compartilhar a configuração que antes ficaria em uma longa linha de docker run.

Compose: banco de dados em outro serviço

Imagem pronta do PostgreSQL + estado em volume nomeado

```
db:
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: aula
    POSTGRES_PASSWORD: aula
    POSTGRES_DB: aula
  volumes:
    - dados:/var/lib/postgresql/data
volumes:
  dados:
```

Configuração inicial

A imagem cria usuário e banco na primeira inicialização de dados vazios.

Persistência

dados é um recurso separado. O caminho pertence ao PostgreSQL 16 deste exemplo.

API e banco podem ser atualizados ou recriados separadamente; os dados não devem depender da camada gravável do banco.

Inicializar o contador não é migrar o banco

init.sql prepara o estado inicial somente em um diretório de dados vazio

```
CREATE TABLE IF NOT EXISTS contador (  
  id integer PRIMARY KEY,  
  total integer NOT NULL  
);  
INSERT INTO contador (id, total)  
VALUES (1, 0) ON CONFLICT (id) DO NOTHING;
```

Estado inicial

Tabela contador e linha id=1.
O total começa em zero num volume novo.

Depois da primeira vez

Editar init.sql não altera automaticamente o banco existente. Mudanças posteriores exigem migração.

Recriar um container e recriar o estado do banco são operações diferentes.

A requisição percorre duas conexões

HTTP do cliente para a API; SQL da API para o PostgreSQL



localhost pertence ao ambiente de quem faz a chamada; dentro da API ele não significa “o container do banco”.

Porta documentada, publicada e interface de escuta

Três decisões distintas precisam ser coerentes

EXPOSE 3000

Metadado da imagem.
Documenta uma porta pretendida.
Não cria acesso pelo host.

8080:3000

Publicação no host.
8080 recebe o cliente.
3000 deve coincidir com a porta real do processo.

HOST=0.0.0.0

A API escuta nas interfaces do container.
127.0.0.1 restringiria a escuta ao loopback dessa instância.

Trocar 8080 por 9090 muda o endereço do cliente; a comunicação da API com db:5432 continua igual.

Configuração de execução fora da imagem

A mesma imagem pode receber valores distintos em cada ambiente

environment

Entrega variáveis ao processo.

Ex.: HOST: 0.0.0.0.

A API lê process.env.HOST.

.env no Compose

Pode fornecer valores para interpolar \${VAR} no YAML.

Não injeta sozinho todo o arquivo no container.

Aplicar mudanças

Variável de execução: recriar a instância.

Código/dependência: reconstruir imagem.

restart não aplica todo YAML alterado.

Após mudar HOST no YAML: `docker compose up -d --no-deps --force-recreate --wait app`.

Recursos para reduzir acesso desnecessário

Configurar de acordo com o que o processo realmente precisa

Usuário e escrita

USER node no Dockerfile.
Bind mount :ro para init.sql.
read_only pode restringir o filesystem da instância.

Segredos e privilégios

Não colocar senhas reais na imagem.
Compose secrets entrega arquivos aos serviços autorizados.
A aplicação precisa lê-los.

Aplicação ao projeto

Declarar diretórios graváveis.
Evitar privileged sem necessidade.
Limitar portas e redes expostas.

Permitir somente o necessário exige conhecer os arquivos, conexões e permissões da aplicação.

Arquivos da instância, volume e bind mount

Escolha onde cada tipo de dado deve viver

Camada gravável

Arquivos da instância.
Remover o container elimina
essa camada.

Volume nomeado

Gerenciado pelo Docker.
O recurso pode persistir
após a recriação.

Bind mount

Caminho do host.
No exemplo: init.sql,
montado somente leitura.

Estado de negócio deve ter persistência planejada; uma imagem nova não deve apagar o banco.

Estado do processo, prontidão e funcionalidade

Defina o que cada evidência realmente mede

RUNNING

Processo em execução.
Não prova acesso ao banco.

HEALTHY

O check configurado passou.
Seu alcance depende do teste.

FUNCIONAL

A operação do usuário funciona.
Ex.: gravar e ler /visitas.

Um estado verde vale somente o que a verificação executada cobre.

Healthcheck: comando e parâmetros de observação

Trecho real do banco no laboratório

```
healthcheck:  
  test: ["CMD-SHELL",  
        "pg_isready -U aula -d aula"]  
  interval: 5s  
  timeout: 3s  
  retries: 10  
  start_period: 10s
```

Quando observar

interval: entre verificações.
timeout: limite por execução.
start_period: tolerância inicial.

Como interpretar

retries: falhas consecutivas.
Saída 0: sucesso do check.
pg_isready não testa a regra de negócio.

O healthcheck da API consulta /health/ready; o teste funcional ainda precisa executar /visitas.

depends_on: coordenar a partida dos serviços

Esperar um check inicial reduz a corrida de inicialização

```
services:
  app:
    # Demais campos definidos anteriormente
    depends_on:
      db:
        condition: service_healthy
```

Partida

Compose espera o check de db passar antes de iniciar app.

Operação

O banco ainda pode cair depois.
A API precisa tratar falhas e timeouts.
Um retry de escrita exige cuidado com duplicação.

Inicialização coordenada ajuda a começar; tratamento de falhas ajuda a continuar operando.

Limites de CPU e memória: controlar o consumo

Exemplo de configuração adicional do serviço app • valores ilustrativos

```
services:  
  app:  
    # Junto dos demais campos do serviço  
    cpus: 0.50  
    mem_limit: 256m  
  
# Observar depois de aplicar a configuração:  
# docker compose stats --no-stream
```

CPU

0.50 limita o tempo de CPU ao equivalente a meia CPU.
Não reserva um núcleo exclusivo.

Memória

256m limita memória.
Uso excessivo pode levar a encerramento por OOM.

Medir → definir limite → testar carga e comportamento; copiar números não é dimensionar.

Políticas de reinício: reagir à saída do processo

No Compose: restart: unless-stopped • escolha conforme a finalidade

Política	Comportamento principal
"no" (padrão)	Não reinicia automaticamente.
on-failure[:N]	Reinicia após saída com erro; N pode limitar tentativas.
always	Reinicia após saída; parada manual suspende até nova partida do daemon ou do container.
unless-stopped	Semelhante a always; preserva a intenção de parada manual mesmo após reinício do daemon.

Restart não é healthcheck, não recupera dados perdidos e não oferece alta disponibilidade entre hosts.

Subir a aplicação integrada e verificar a resposta

Usar o compose.yaml completo fornecido no laboratório

```
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
curl -fsS http://localhost:8080/health/live
curl -fsS http://localhost:8080/health/ready
curl -fsS -X POST http://localhost:8080/visitas
curl -fsS http://localhost:8080/visitas
```

Preparar

Engine ativo; porta 8080 livre.
Terminal na pasta do laboratório.
config valida a declaração.

Observar

up cria serviços e recursos.
--wait aguarda os checks.
As chamadas HTTP verificam a resposta.

A API só está integrada quando a operação real consegue atravessar a conexão com o banco.

Interpretar as saídas da demonstração

Exemplo de sequência • números ilustrativos, dependentes do estado do volume

Observação	Interpretação
GET /health/live → 200	O processo HTTP respondeu.
GET /health/ready → 200	A consulta básica no banco funcionou.
GET total 7 → POST total 8 → GET total 8	A operação gravou e leu o incremento.
Após down/up, GET total 8	O estado foi preservado no volume reutilizado.

Comandos e resultados devem ser relacionados a uma afirmação verificável sobre o projeto.

Comprovar persistência com uma experiência

Anotar → recriar containers → ler novamente

```
curl -fsS -X POST http://localhost:8080/visitas
curl -fsS http://localhost:8080/visitas
docker compose down
docker compose up -d --wait
curl -fsS http://localhost:8080/visitas
```

Antes

Leia e anote o total N.
POST deve produzir N + 1.
Guarde o valor observado.

Depois

down sem -v preserva o volume nomeado.
Com o mesmo projeto, up o reutiliza.
GET deve devolver o total preservado.

Persistência além da instância não é recuperação de desastre; volume e backup resolvem problemas diferentes.

Verificação | Rede, dados e saúde

Verificação do bloco • responda com os conceitos e exemplos que acabamos de estudar

1. Situação

A porta do host muda para 9090. A API deve passar a usar db:9090?

2. Situação

Depois de down/up sem -v, o total permanece. Isso prova que há backup?

3. Situação

O check SELECT 1 passa, mas a tabela contador não existe. /visitas está garantido?

Discuta primeiro. O próximo slide traz as respostas comentadas.

Respostas comentadas | Rede, dados e saúde

Gabarito explicado • relacione cada resposta ao exemplo observado

1. Resposta

Não. 9090 é a entrada do cliente na API. A conexão interna com o banco continua db:5432.

2. Resposta

Não. Prova persistência no volume reutilizado. Backup requer uma cópia recuperável e restauração testada.

3. Resposta

Não. A consulta básica pode passar enquanto a operação falha por schema ausente. Teste GET/POST /visitas.

Uma resposta completa identifica o componente envolvido e explica a evidência.

Diagnóstico: observar antes de alterar

Uma sequência de verificação para localizar a falha

1 • Fase

Build ou
execução?

2 • Estado

ps -a

3 • Evidência

logs

4 • Integração

Rede e banco

5 • Prova

Requisição real

Sintoma → hipótese → teste → correção mínima → nova validação → registro.

Comandos com uma pergunta técnica definida

Cada comando produz uma evidência diferente

Comando	O que ele ajuda a responder
<code>docker compose ps -a</code>	A instância iniciou ou encerrou?
<code>docker compose logs --tail=60 app</code>	Qual erro a aplicação emitiu?
<code>docker compose exec db pg_isready -U aula -d aula</code>	O banco aceita conexões?
<code>docker compose exec app node -p "process.env.HOST"</code>	Qual bind a API recebeu?
<code>docker compose config --quiet</code>	A declaração é válida para o Compose?

Não memorize uma lista de comandos sem entender qual hipótese cada um testa.

Caso resolvido: banco saudável, API retorna 503

Falha provocada: hostname localhost na conexão da API

Sintoma e evidência

db está healthy.
ready retorna 503.
Log da API: conexão recusada.
URL aponta para localhost:5432.

Causa explicada

A conexão sai da API.
localhost aponta para ela mesma.
O banco está no serviço db da rede compartilhada.

Correção e prova

Trocar localhost por db.
Recriar app com a nova configuração.
Repetir ready e GET/POST /visitas.

A evidência antes/depois deve confirmar a causa; apagar o volume não testa esta hipótese.

Caso resolvido: check verde, acesso externo falha

Falha provocada: HOST=127.0.0.1 dentro do container

Sintoma e evidência

Chamada interna em localhost:3000 funciona.

Cliente no host:8080 falha.

O healthcheck interno pode ficar verde.

Causa explicada

A API escuta apenas no loopback da instância.

O tráfego publicado chega pela interface de rede do container.

Correção e prova

Restaurar HOST=0.0.0.0.

Recriar app.

Comparar novamente a mesma rota de dentro e de fora.

Um teste interno bem-sucedido não garante o caminho percorrido pelo cliente externo.

Caso resolvido: dependência cai durante o uso

Parar apenas o banco diferencia processo vivo de funcionalidade disponível

Provocar e observar

`docker compose stop db`
live continua 200.
ready e `/visitas` retornam 503.

Explicação

A API segue em execução.
A operação precisa de consulta SQL.
`depends_on` já cumpriu seu papel na partida.

Recuperar e provar

`docker compose start db`
Aguardar o banco aceitar conexões.
ready volta a 200; o total permanece.

Recuperar a dependência e testar novas requisições pode bastar; a decisão deve seguir a evidência.

IA no diagnóstico: hipótese com teste verificável

Exemplo de pedido depois de coletar o sintoma e a configuração relevante

Contexto fornecido

API retorna 503 em ready.
db aparece healthy.
URL usa localhost:5432.
Sem senhas nem dados reais.

Pedido à IA

“Liste hipóteses priorizadas.
Para cada uma, dê um teste não destrutivo e o resultado que a descartaria.”

Validação humana

Conferir a relação causa/ação.
Executar a verificação.
Rejeitar down -v para um erro de hostname.
Registrar correção e resultado.

Toda saída de IA é hipótese até ser validada por teste, execução ou revisão humana.

Como este encontro entra no projeto integrador

O mesmo projeto evolui do código validado ao ambiente executável

E1-E3 | BASE

Projeto escolhido
Colaboração e código
Testes e pipeline CI



E4 | EXECUÇÃO

Imagem da aplicação
Serviços integrados
Diagnóstico com evidência



E5-E6 | ENTREGA

Configuração e promoção
Rollback e governança
Demonstração final

E4 alimenta os 20% de Containers/Compose já previstos na rubrica; os demais pesos permanecem os oficiais.

Traduzir o exemplo para a aplicação da equipe

A linguagem pode mudar; as responsabilidades continuam

No exemplo da aula	No projeto da equipe
Node + server.js	Runtime e comando reais da aplicação.
pg + lockfile npm	Dependências controladas da stack escolhida.
app → db	Componentes que de fato precisam conversar.
dados + init.sql	Persistência e inicialização/migração pertinentes.
live, ready e /visitas	Check de saúde e operação funcional do projeto.

Antes de escrever o Dockerfile, anote comando, runtime, dependências, configuração e estado do próprio projeto.

Plano técnico: decisões antes de executar

Preencher no README da própria equipe

Imagem

Qual comando inicia a aplicação?

Qual runtime e versão ela usa?

Quais arquivos precisam entrar?

Integração

Quais serviços existem?

Quem chama quem e por qual porta?

Quais variáveis são obrigatórias?

Dados e validação

Onde o estado deve persistir?

O que o check de saúde mede?

Qual operação comprova funcionamento?

As decisões do README devem corresponder aos arquivos versionados e à execução que será demonstrada.

Laboratório: aplicar ao projeto da equipe

70 minutos reservados para implementação e evidências

0–20 min | Empacotar

Dockerfile e .dockerignore.
Dependências controladas.
Imagem inicia com comando claro.

20–45 min | Integrar

Compose com serviços pertinentes.
Configuração, rede, dados e saúde.
Executar uma operação funcional.

45–70 min | Validar

Provocar e corrigir uma falha.
Conferir README a partir de clone limpo.
Registrar comandos, resultados e decisões.

A equipe deve conseguir explicar o que cada serviço faz e reproduzir sua própria execução.

Critérios de aceite: o que deve ficar demonstrado

Evidências no repositório • critério Containers/Compose: 20%

Reprodução

Dockerfile, Compose e arquivos necessários.

README com setup, subida, teste e parada.

Nenhum arquivo local oculto indispensável.

Comportamento

Aplicação inicia.

Operação alcança a dependência.

Estado persiste quando necessário.

Check tem um contrato explicado.

Raciocínio

Falha reproduzida e causa confirmada.

Correção validada por execução.

Registro crítico de IA, se utilizada.

Escolhas justificadas.

Arquivo presente é parte da evidência; resultado reproduzível e explicação da decisão completam a entrega.

Exemplo preenchido: evidência de uma falha corrigida

Modelo ilustrativo para adaptar aos comandos e resultados reais da equipe

Campo	Exemplo preenchido
Sintoma	GET /health/ready devolvia HTTP 503.
Hipótese e teste	URL usava localhost; db estava saudável. Conferimos destino e logs.
Causa	A API procurava o banco em sua própria instância.
Correção	Hostname mudou para db; somente app foi recriado.
Validação	ready retornou 200; POST incrementou N para N+1 e GET confirmou.

Substitua os valores do exemplo pela execução real e mantenha toda evidência no repositório.

Verificação final | Explicar o próprio projeto

Verificação do bloco • responda com os conceitos e exemplos que acabamos de estudar

1. Situação

Ao apresentar o projeto, como distinguir imagem, container e os dois sentidos de runtime?

2. Situação

Como provar que sua aplicação está integrada à dependência, além de mostrar o container Up?

3. Situação

Qual evidência liga uma alteração de configuração à correção de uma falha?

Discuta primeiro. O próximo slide traz as respostas comentadas.

Respostas comentadas | Explicar o próprio projeto

Gabarito explicado • relacione cada resposta ao exemplo observado

1. Resposta

Imagem: artefato.
Container: instância.
Runtime da linguagem executa o código; runtime de containers prepara e inicia processos isolados.

2. Resposta

Executar uma operação funcional com resultado esperado; verificar saúde e persistência conforme o contrato do projeto.

3. Resposta

Sintoma reproduzido, hipótese testada, mudança específica e repetição do mesmo teste com resultado corrigido.

Uma resposta completa identifica o componente envolvido e explica a evidência.

Do ambiente executável à entrega controlada

Conexão com o Encontro 5 e a apresentação final

Levar pronto

Projeto executável por comando documentado.
Integração demonstrada.
Falha corrigida e registrada.

Preparar a entrega

Identificar a imagem validada.
Separar configuração de ambiente.
Definir promoção e rollback.

Apresentação final

Explicar o fluxo código → CI → imagem → execução.
Mostrar evidências reais.
Defender as escolhas adotadas.

Containerização tem valor quando melhora a reprodução, a integração e a entrega do projeto.

Referências e consulta depois da aula

Documentação oficial e material do próprio laboratório

Conceitos

Docker: visão geral e containers.

Engine e Compose.
containerd e runc:
responsabilidades.

Prática

Dockerfile, cache
e .dockerignore.

Redes, volumes e health
checks.

Imagem oficial PostgreSQL.

Material disponível

Notas do apresentador.

Guia completo por slide.

README e exemplos
executáveis.

Gabaritos das verificações.

Fim da trilha principal. Os próximos slides são material de consulta, sem nova entrega obrigatória.

Consulta | Registry, tag e digest

Identificar o artefato que será promovido

Registry

Armazena e distribui imagens.

push publica; pull obtém.

Não executa a aplicação.

Tag

Nome legível de uma referência.

Pode mudar de alvo.

latest não significa conteúdo imutável.

Digest

Identifica conteúdo.

Permite registrar o artefato validado.

Em imagens multiplataforma, registre também a plataforma.

Mesma tag não prova igualdade de conteúdo entre máquinas ou momentos diferentes.

Consulta | Multi-stage e separação de responsabilidades

Exemplo conceitual de aplicação TypeScript • não é necessário na API do laboratório

Estágio de build

Instala dependências de desenvolvimento.

Executa o compilador.

Gera o diretório dist/.

Estágio de execução

Recebe dist/ e dependências de produção.

Executa com Node.

Não precisa do compilador só para iniciar.

Decisão

Inclua somente o necessário para executar.

Verifique bibliotecas nativas e plataforma.

Não presuma ganho de segurança só pelo número de estágios.

Multi-stage é uma escolha de empacotamento; use quando houver uma separação útil entre construir e executar.

Consulta | Verificar a imagem no CI

Evolução do pipeline já criado no Encontro 3

Validar o código

Instalação controlada.
Testes da aplicação.
Build quando a stack exige.

Validar a execução

Construir a imagem.
Subir dependências de teste.
Executar uma operação integrada.

Registrar e encerrar

Identificar o artefato testado.
Guardar logs pertinentes.
Encerrar apenas os recursos do job, inclusive em falha.

CI verde só comprova o que foi testado; executar a imagem amplia a evidência sobre o artefato.

Consulta | Volume, bind mount e tmpfs

Três maneiras de montar arquivos, com finalidades diferentes

Montagem	Quando usar	O que lembrar
Volume nomeado	Dados que precisam sobreviver à instância.	Precisa de backup; existe fora da camada gravável.
Bind mount	Arquivo/diretório específico do host.	Depende de caminho e permissões do host.
tmpfs (Linux)	Arquivos temporários durante a execução.	Não persiste após parada; pode haver swap.

Persistência, compartilhamento com o host e armazenamento temporário são requisitos diferentes.

Consulta | Diferentes modos de rede

O laboratório usa a rede bridge criada pelo Compose

Driver	Finalidade e contexto
bridge	Conecta containers no mesmo host; redes definidas pelo usuário permitem descoberta por nome.
host	Compartilha a pilha de rede do host; comportamento e suporte dependem da plataforma.
none	Desativa a conectividade externa da instância; mantém loopback.
overlay	Rede entre hosts, com requisitos de Swarm; fora do laboratório.
macvlan / ipvlan	Integração especializada com a rede física, exigindo planejamento de endereçamento.

Escolha a topologia necessária antes do driver; uma rede compartilhada não publica sozinha portas no host.

Consulta | Profiles: serviços opcionais no Compose

Exemplo de ferramenta administrativa que não precisa iniciar sempre

```
services:
  admin:
    image: adminer:4
    profiles: ["ferramentas"]
    ports:
      - "127.0.0.1:18082:8080"
```

```
# Na configuração completa com app e db:
# docker compose --profile ferramentas up -d
```

Sem o profile

up inicia serviços sem profile.
admin fica fora desse conjunto.

Com ferramentas

Inclui admin no conjunto.
O exemplo deve compartilhar a rede do db.

Profiles permitem escolher ferramentas ou tarefas opcionais sem duplicar toda a aplicação.

Consulta | Overrides e Watch no desenvolvimento

Adaptar o ambiente local mantendo a configuração compreensível

Overrides

Arquivos adicionais combinam configurações.

Ex.: `compose.yaml` + `compose.dev.yaml`.

Revise o resultado com `compose config`.

Watch

Regras `develop.watch` podem sincronizar arquivos ou reconstruir a imagem.

O efeito depende da ação configurada.

Limite do recurso

Sincronizar arquivo não garante recarga do processo.

Framework e comando precisam suportar o fluxo escolhido.

Confira a versão do Compose e a configuração resultante; conveniência local não dispensa testar a imagem final.

Consulta | BuildKit, Buildx e cache de construção

Construir com eficiência e escolher a plataforma de destino

BuildKit

Backend de construção.
Pode reutilizar resultados e usar cache mounts.
Nem todo build será executado do zero.

Buildx

Interface para recursos de build.
Permite escolher builders e plataformas.
Ex.: linux/amd64 ou linux/arm64.

Exemplo e cuidado

Uma biblioteca nativa precisa ser compatível com a plataforma final.
Build bem-sucedido pede teste de execução.

Otimizar a construção e controlar a plataforma complementam a reprodução; não substituem testes.

Consulta | Outros recursos do ecossistema

Conhecer o propósito sem exigir todas as ferramentas no projeto

Recurso	Para que serve
Docker Hub / registry	Distribuição de imagens, tags e controle de acesso.
Contexts	Selecionar qual Engine a CLI controla; conferir antes de operar.
Docker Scout	Análise de imagens e informações sobre vulnerabilidades.
Swarm mode	Orquestração de serviços entre nós; difere do Compose local.
Desktop Dashboard	Interface para visualizar e operar recursos do ambiente local.

A entrega desta aula usa os recursos necessários ao projeto; conhecer opções não cria novas exigências.